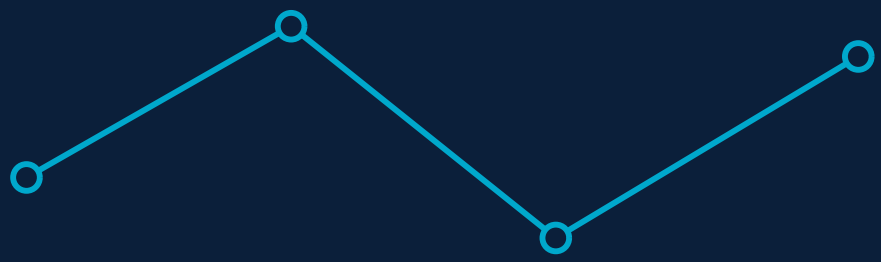
NOVEX 2026 | FINAL PROJECT REPORT

SMART SHARE

Intelligent, location-based cab pooling for Mumbai



Same Direction. Similar Timing. One Shared Ride.

Team name: [ENTER TEAM NAME]

Team members: [ENTER NAMES AND ROLES]

College / institution: [ENTER COLLEGE]

Mentor: [ENTER MENTOR, IF ANY]

Submission date: [ENTER DATE]

Status statement: matching suggestions are implemented; ride acceptance, booking and payments are not.

Contents

1. Abstract
2. Problem Statement
3. Proposed Solution
4. Objectives
5. Key Features
6. Technology / Tech Stack
7. System Architecture
8. Implementation
9. Results / Screenshots
10. Innovation
11. Impact
12. Future Scope
13. References

> **Reading guide.** This report separates **intended design** from **what has been built and verified**. Every feature carries one of three labels: **Implemented and tested**, **Implemented, awaiting complete end-to-end verification**, or **Planned**. Test figures come from the project's own verification reports; nothing here is a measured user outcome.

Section 1 | Abstract

Smart Share is a mobile-first platform that helps people discover others whose journeys are compatible with theirs, so that they can share a cab instead of booking separate ones. People leaving the same station, airport, event or business district at about the same time, heading in a similar direction, rarely know about each other. Smart Share lets a user post a *ride request* (pickup, destination, departure window, seats, and the maximum extra minutes they accept) and returns a ranked list of compatible requests with an explanation of why each was suggested.

The matching engine is location-aware and time-aware. It uses **PostGIS** to shortlist nearby pickups, then checks departure-window overlap, travel direction and seat capacity. It evaluates the valid pickup and drop-off orders for a pair, using **OSRM** road-network durations (with a clearly labelled approximate fallback), and keeps only pairings that stay inside each rider's detour limit. The stack is React, TypeScript and Vite on the front end; Python, FastAPI and SQLAlchemy on the back end; PostgreSQL with PostGIS hosted on Supabase; and Supabase Auth with backend JWT verification.

Expected benefits are lower individual fares, better seat utilisation and fewer separate trips. These are **expected** benefits and have not been measured. Smart Share currently produces ranked *suggestions*; it does not book cabs, assign drivers or process payments.

Section 2 | Problem Statement

In dense cities such as Mumbai, many people start journeys from the same place, at nearly the same time, towards the same or neighbouring destinations. Yet each person typically books a separate cab, because there is no easy way to find out who else is going their way.

Typical situations:

- **Railway stations** (for example Dadar or CSMT): many commuters leave the same exit within minutes of each other.
- **Airports** (for example Mumbai T2): arriving passengers head to nearby suburbs in a similar direction.
- **Events and functions**: a college festival or wedding ends, and hundreds of guests leave at once.
- **Busy commercial areas** (for example BKC): office crowds travel to similar residential corridors in the evening.

Informal coordination through messaging groups is manual and unreliable. Even when two travellers find each other, they cannot easily judge whether sharing is practical: how far apart the pickups are, whether their departure times line up, whether one rider would face a long backtracking detour. Straight-line closeness is misleading in Mumbai; the project design notes show that two pickups at the same station can head in opposite directions (Dadar to Colaba versus Dadar to Borivali), and that water bodies make nearby points far apart by road.

The consequences are **unnecessary separate journeys**, **higher individual costs**, **underused vehicle capacity** and added congestion near crowded venues.

Section 3 | Proposed Solution

Smart Share connects riders through **ride requests**. A user signs in, creates a request, and asks the system to find compatible requests from other users.

- **Create a ride request.** Choose a pickup and a destination (the current UI offers Mumbai landmark presets such as BKC, Airport T2, Powai, Lower Parel, Dadar and Andheri West).
- **Select a departure window** (earliest and latest acceptable departure).
- **State seats needed** and the **maximum extra minutes** the rider will accept.
- **Find matches.** The backend discovers candidates, filters them and evaluates routes.
- **Review ranked suggestions** with score factors, each rider's estimated detour, the stop order and a routing-accuracy notice.

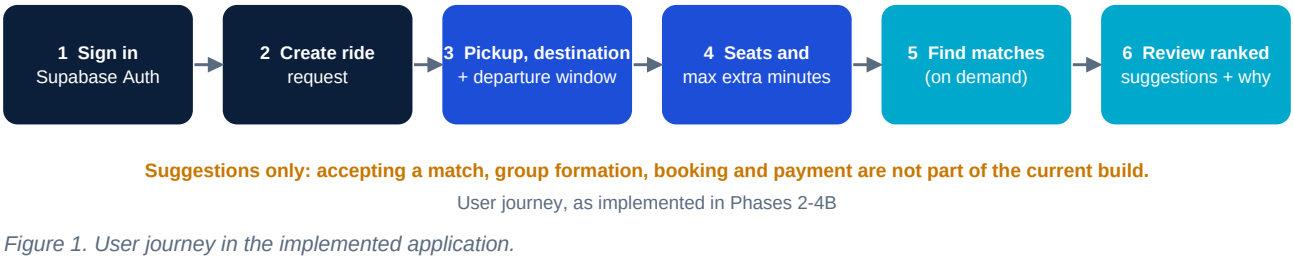


Figure 1. User journey in the implemented application.

How candidates are found: the engine shortlists active, unexpired requests from other users within 2 km of the pickup, with overlapping departure windows and combined seats of at most four. It then rejects pairs travelling in clearly different directions, evaluates the valid pickup/drop-off orders using road-network durations, and discards any order in which a rider's extra time exceeds their own limit. Survivors are scored and the top five are returned.

Scope note. The documentation does not describe automatic cab booking, driver assignment or payments as implemented, and this report does not claim them.

Section 4 | Objectives

Technical objectives

- Discover compatible passengers with **location-aware** spatial queries (PostGIS geography, GiST index).
- Apply **time-aware** matching through departure-window intersection.
- Evaluate **direction and destination compatibility**, and reject backtracking pairings.
- Evaluate valid **pickup/drop-off orders** and keep each rider's **detour** within their stated limit.
- Keep results **explainable**, deterministic and clearly labelled where routing is approximate.
- Establish a **scalable foundation**: separated modules, replaceable routing provider, migrations and automated tests.

User-impact objectives

- Make it easy to find fellow travellers, and reduce reliance on separate individual cab journeys.
- Improve use of available seats.
- Minimise unnecessary detours for every rider.
- Offer an accessible, **mobile-first** experience with a simple request form.
- Help riders understand *why* a suggestion was made.

> The user-impact objectives are design goals. The project's original pilot targets (for example 30% of eligible requests matched, 20% estimated savings) have **not** yet been tested and are not reported as achieved.

Section 5 | Key Features

Status is taken from the latest implementation and verification reports. 'Tested' means automated tests, plus live checks where noted.

Feature	Planned	Implemented	Tested	Status
Supabase Auth with backend JWT verification (ES256/JWKS)	Yes	Yes	Auth test suite (18 tests)	Implemented and tested
User profiles (retrieve/update)	Yes	Yes	API tests	Implemented and tested
Ride request create, list, view, edit, cancel; ownership checks	Yes	Yes	API + model tests; UI tests	Implemented and tested
Pickup/destination (landmark presets), departure window, seats, detour preference	Yes	Yes (presets; free map picker not documented)	UI tests	Implemented and tested
PostGIS candidate filtering (2 km radius, window overlap, active/unexpired, LIMIT 20)	Yes	Yes	Unit tests; GiST usability and live PostGIS run (rolled-back transaction)	Implemented and tested; browser E2E pending
Direction and seat-capacity checks	Yes	Yes	Unit tests	Implemented and tested
Pairwise stop-order evaluation and per-rider detour limits	Yes	Yes	Unit tests	Implemented and tested
Weighted scoring, ranking (top 5) and match explanations	Yes	Yes	Unit, API and UI tests	Implemented; authenticated hosted E2E pending
OSRM road-network durations and approximate fallback	Yes	Yes	Adapter and fallback tests; one real BKC to Powai call	Implemented and tested; public OSRM demo is best-effort
'Find matches' UI with loading, error, empty and ranked states	Yes	Yes	Frontend tests	Implemented; authenticated hosted E2E pending
Match acceptance / decline, ride groups, proposal expiry	Yes	No	-	Planned (Phase 7)
Fare estimation and allocation	Yes	No	-	Planned (Phase 5)
Map visualisation, notifications, events, payments, live tracking, cab booking	Yes / Future	No	-	Planned or future

Table 1. Feature-status matrix.

Scoring model (as implemented)

Composite score = 30% pickup proximity + 30% destination similarity and direction + 20% departure-time compatibility + 20% individual detour headroom. Seat capacity is a hard eligibility gate, not a scoring factor. Ties are broken by lower total detour, then smaller pickup distance. Early design drafts used five factors; the weights above are the implemented ones.

Section 6 | Technology / Tech Stack

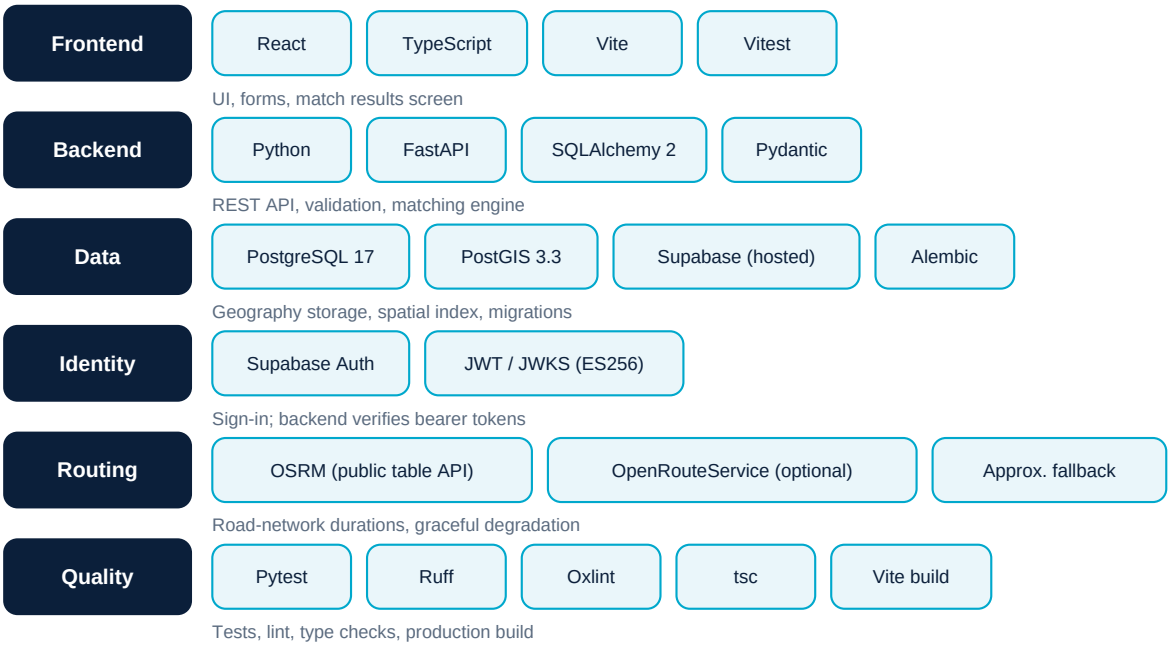


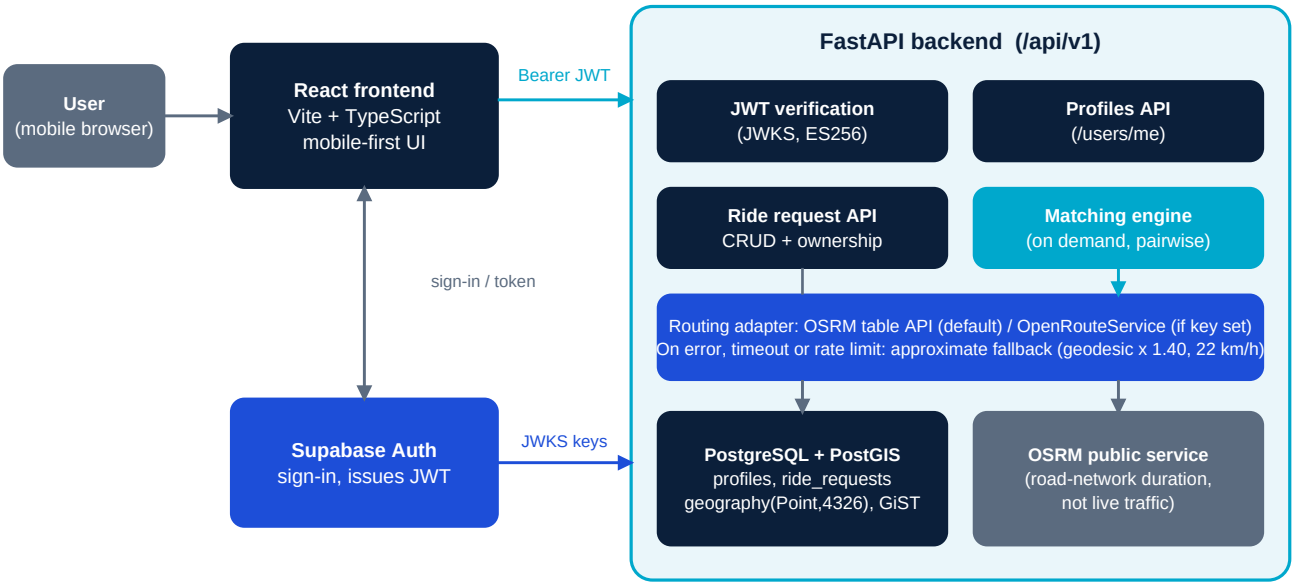
Figure 2. Technology stack, limited to components documented as implemented.

Technology	Purpose in Smart Share	Why it fits
React, TypeScript, Vite	Single-page, mobile-first UI and ride-request forms	Typed components, fast builds and a rich ecosystem
Python, FastAPI	REST API under /api/v1 and the matching engine	Async support, automatic OpenAPI docs, clean dependency injection
Pydantic	Request/response validation and schemas	Rejects invalid coordinates, windows and seats early
SQLAlchemy 2 (async)	Database access and models	Async queries and parameterised SQL
PostgreSQL + PostGIS	Stores requests as geography(Point, 4326); spatial radius search	ST_DWithin with a GiST index narrows candidates quickly
Supabase (hosted)	Managed PostgreSQL and authentication	No self-managed infrastructure for a hackathon team
Alembic	Versioned schema migrations (revision 5b69062ab3ef)	Reproducible, reviewable schema changes
Supabase Auth, JWT/JWKS	User sign-in; backend verifies bearer tokens	Identity is derived from the token, never from client input
OSRM (+ approximate fallback)	Road-network travel durations for detour checks	No credential needed; fallback keeps matching available
Pytest, Ruff, Oxlint, tsc, Vitest	Backend tests/lint; frontend lint, type check and tests	Catch regressions and keep code quality consistent

Table 2. Technologies and rationale.

Not claimed as implemented: Tailwind CSS, shadcn/ui, Leaflet maps, TanStack Query, Playwright and Docker-based infrastructure appear in the original architecture proposal, but the verification reports reviewed here do not confirm them. Please confirm against the repository before listing them.

Section 7 | System Architecture



Primary data flow

User > React frontend > Supabase Auth > authenticated API request > FastAPI > matching engine > PostGIS/PostgreSQL > OSRM routing (or approximate fallback) > ranked proposals with explanations > JSON response > frontend

Candidate details exposed to the client omit contact and auth data. Proposals are computed on demand and are not persisted.

Figure 3. System architecture and primary data flow.

The frontend never decides feasibility: the backend is authoritative. The routing module sits behind a provider interface, so OSRM, OpenRouteService or the fallback can be swapped without changing the matching logic. Proposals are recomputed on each request, so cancelling or expiring a request removes it from later results without extra bookkeeping.

Steps 1, 2 and 4 run inside one parameterised SQL query; steps 3 and 5-8 run in Python.

Hard filters run before ranking: a high score can never make an infeasible pairing eligible.

Figure 4. Matching workflow.

Section 8 | Implementation

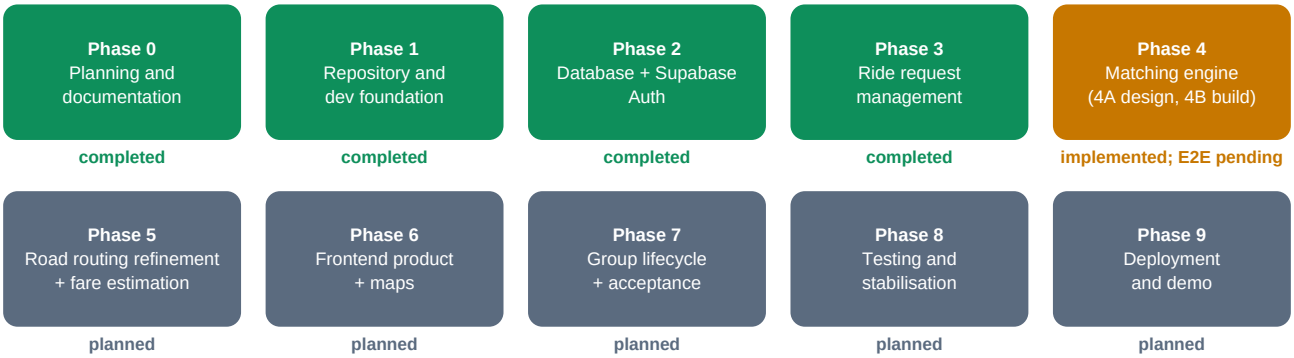


Figure 5. Implementation timeline. Phases 5-9 are planned.

Phase 0 - Planning and documentation

Six foundation documents: project charter, SRS/PRD, architecture and technical design, database and API contracts, roadmap, and developer guide. Together they froze the MVP scope and set the matching approach as a rule-based, explainable heuristic.

Phase 1 - Foundation

Repository, React/Vite/TypeScript frontend skeleton, FastAPI backend skeleton, environment configuration and a health endpoint. The roadmap records a running local environment and CI configuration. *The detailed Phase 1 audit was not among the documents reviewed, so details such as the Docker/PostGIS setup should be added by the team.*

Phase 2 - Database and Supabase integration

SQLAlchemy models and the first Alembic migration (`5b69062ab3ef`) create profiles and ride requests on hosted PostgreSQL 17.11 with PostGIS 3.3.7. Locations are stored as geography(Point, 4326) with GiST indexes, and database constraints enforce valid windows, 1-6 seats, detour limits and valid status values. Supabase Auth is integrated, and the backend verifies ES256 tokens through JWKS.

Phase 3 - Ride request management

Authenticated create, list, view, update and cancel endpoints with ownership checks, coordinate and window validation, and matching React forms and listings. Requests move through ACTIVE, MATCHED, CANCELLED, EXPIRED and COMPLETED states.

Phase 4 - Matching engine (4A design, 4B build)

Phase 4A specified the algorithm and its worked Mumbai examples. Phase 4B implemented candidate discovery, direction and capacity gating, enumeration of pairwise stop orders, per-rider detour checks, scoring, ranking, three matching endpoints, the routing adapters and the 'Find matches' UI.

Technical challenges and how they were addressed

Challenge	Resolution
Straight-line distance misleads in Mumbai (water barriers, opposite headings from one station)	Direction gate (hard reject above 90 degrees) and road-network durations for detours
Two precedence-valid stop orders in which riders never share the cab	Engine enumerates six orders and rejects the two non-sharing ones
Public routing service is rate-limited and has no uptime guarantee	Bounded requests (4 points), 1.05 s spacing, timeout, labelled approximate fallback
Backend test suite stalled on an existing test (event-loop incompatibility on Python 3.14)	Tests switched to a uvloop loop factory; no test skipped or timeout raised

Challenge	Resolution
Post-merge audit: real PostGIS returned raw hex EWKB for geography columns, crashing coordinate parsing	Query now selects ST_AsText; parser also decodes EWKB/WKB; regression tests added
Spatial index use at tiny data volume	Verified the predicate is GiST-compatible; planner chose the temporal index on a two-row table

Table 3. Key challenges.

Section 9 | Results / Screenshots

No screenshots were available in the documents reviewed, so labelled placeholders mark where genuine captures must be inserted. No mock-ups are shown.

[SCREENSHOT PLACEHOLDER] 1. Homepage
Landing screen of the running app, mobile viewport.

[SCREENSHOT PLACEHOLDER] 2. Authentication screen
Sign-in / sign-up page (hide any email addresses).

[SCREENSHOT PLACEHOLDER] 3. Ride request creation form
Form with pickup, destination, window, seats, extra minutes.

[SCREENSHOT PLACEHOLDER] 4. Active ride request listing
List of requests with the 'Find matches' button visible.

[SCREENSHOT PLACEHOLDER] 5. Ride request details / edit
Detail or edit view of one request.

[SCREENSHOT PLACEHOLDER] 6. Matching interface
Loading or results screen after 'Find matches'.

[SCREENSHOT PLACEHOLDER] 7. Ranked candidate results
Expanded 'Why this suggestion?' with scores and stop order.

Documented testing results

Evidence	Result	Source
Backend test suite	**100** tests: 99 passed and 1 skipped offline; 100 passed with live database	Post-merge verification (Nov 7, 2026)
Backend, earlier run	90 passed, 0 failed, 0 skipped	Phase 4B remediation report
Frontend tests	**28** passed (earlier run: 26)	Post-merge verification; remediation report
Static checks and build	Ruff, Oxlint, TypeScript and Vite production build all passed	Both reports
Real OSRM call, BKC to Powai	645.1 s and 8,803.9 m, road-network based, not live traffic	Remediation report
PostGIS GiST index	idx_ride_requests_pickup_gist exists; spatial predicate is index-compatible	Remediation and post-merge reports
Live pipeline run on hosted PostGIS	1 candidate found, pickup 45.82 m, score 0.986, all inside a rolled-back transaction	Post-merge verification

Table 4. Test evidence. The figures differ between reports because the later post-merge audit added tests; the later numbers are the current ones.

Verification status, stated transparently

Phase 4B verdict: CONDITIONAL PASS. The Phase 4B remediation report could not complete the authenticated hosted end-to-end workflow (real sign-in, frontend-triggered matching request, hosted PostGIS discovery and rendered proposal). The hosted development database had **zero active ride requests** and no two development users were available for the test.

A later post-merge audit found and fixed a spatial-parsing defect and verified the backend matching pipeline against the live hosted PostGIS inside a rolled-back transaction, and it rated the build ready for Phase 5. The reviewed documents still do **not** record the full authenticated browser-driven flow with two users. This report therefore does not claim complete hosted end-to-end verification.

Also unmeasured: performance at realistic data volumes (the hosted tables held only a handful of rows), and any fare, savings or emissions outcome.

Section 10 | Innovation

Smart Share's contribution is not simply splitting a cab fare. It is **deciding which strangers can practically share a journey**, and showing why. It combines several checks that simple proximity matching lacks:

- **Location-aware** pickup discovery using spatial indexing.
- **Time compatibility** through shared departure-window overlap.
- **Direction and destination compatibility**, so a rider heading north is never paired with one heading south.
- **Multiple valid pickup/drop-off sequences**, with riders required to share the cab for part of the trip.
- **Detour-aware feasibility**: each rider's extra time is measured against their own limit.
- **Capacity constraints** as a hard gate.
- **Ranked, explainable results**, with honest labels when routing is approximate.

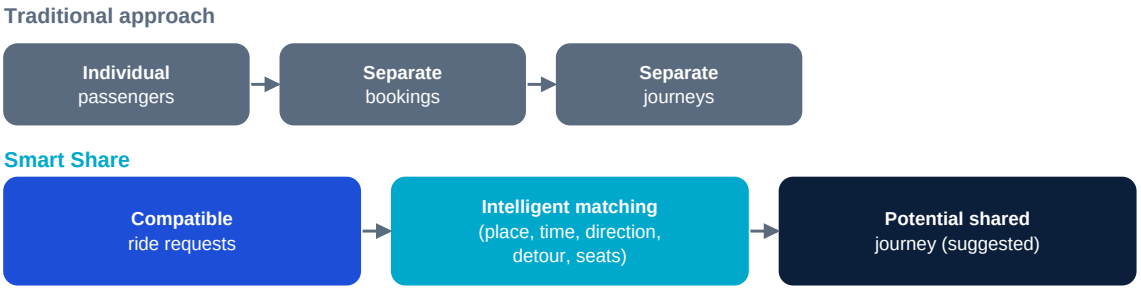


Figure 6. Traditional approach versus Smart Share.

This report makes no claim that Smart Share is the first or only ride-pooling idea. Its focus is a transparent, constraint-driven matching engine designed for Mumbai's geography.

Section 11 | Impact

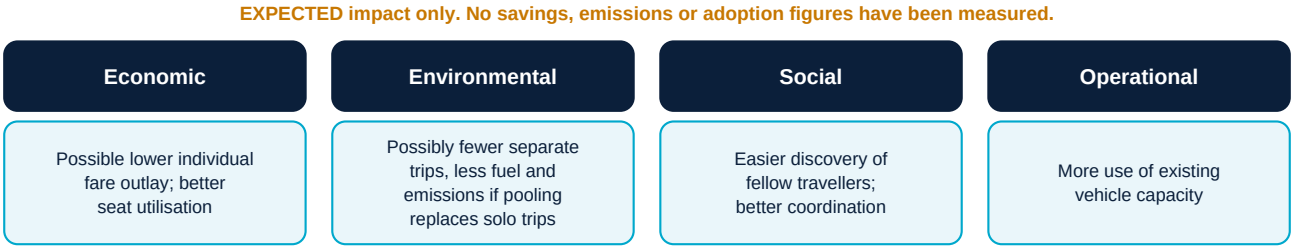


Figure 7. Expected impact (not measured).

- **Economic:** sharing a cab could reduce what each rider pays, and fills seats that would otherwise travel empty.
- **Environmental:** where pooling replaces separate trips, fewer vehicle trips could mean less fuel use and fewer emissions.
- **Social:** easier discovery of fellow travellers and simpler coordination, potentially making transport more accessible.
- **Operational:** better use of existing vehicle capacity.

> None of these benefits has been measured. Realising them depends on user adoption, on fare logic still to be built, and on matches converting into real shared trips, which the current version does not handle.

Section 12 | Future Scope

All items below are **proposed** and are not implemented.

Priority	Item	Roadmap link
Near term	Complete the authenticated hosted end-to-end verification with two development users	Phase 4B acceptance
Near term	Road-distance normalisation, fare estimation and fare splitting	Phase 5
Next	Match acceptance and decline, passenger coordination, proposal expiry, shared ride group formation with transaction-safe seat allocation	Phase 7
Next	Notifications and map visualisation of routes	Phases 6-7
Later	Traffic-aware routing and a managed or self-hosted routing service	Production readiness
Later	Advanced matching optimisation (groups of three or four riders)	Matching extension
Later	Event-based pooling; railway and airport integrations	Roadmap 'should have'
Later	Cab-provider or booking integration, payments, real-time tracking	Out of MVP scope
Later	Production deployment and scaling, load testing at realistic data volumes	Phases 8-9

Section 13 | References

- React documentation: https://react.dev
- Vite: https://vite.dev
- TypeScript documentation: https://www.typescriptlang.org/docs/
- FastAPI documentation: https://fastapi.tiangolo.com
- Pydantic documentation: https://docs.pydantic.dev
- SQLAlchemy documentation: https://docs.sqlalchemy.org
- Alembic documentation: https://alembic.sqlalchemy.org
- PostgreSQL documentation: https://www.postgresql.org/docs/
- PostGIS documentation: https://postgis.net/documentation/
- Supabase documentation: https://supabase.com/docs
- OSRM Table service API: https://project-osrm.org/docs/v5.24.0/api/#table-service
- OSRM demo server policy: https://github.com/Project-OSRM/osrm-backend/wiki/Demo-server
- pytest: https://docs.pytest.org
- Vitest: https://vitest.dev
- Ruff: https://docs.astral.sh/ruff/

Project design documents (internal): Project Charter; SRS/PRD; System Architecture and TDD; Database Schema and API Contracts; Project Roadmap; Operations and Developer Guide; Phase 4A matching-engine design; Phase 4B matching algorithm, implementation, testing and remediation reports; Phase 4B post-merge verification report.